Deep Learning in Computer Vision· Lecture 5 of 9

From AlexNet to VGG and Inception

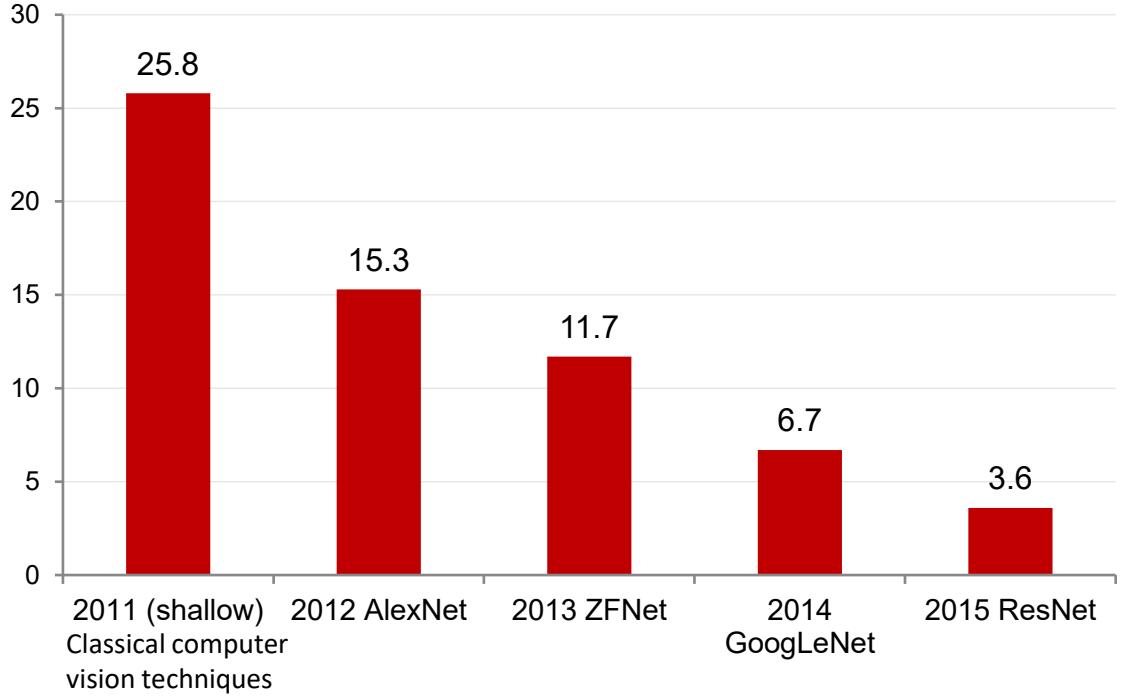
What each architecture added, and why it helped

Recap and Question

- Lecture 4: conv, ReLU, pooling, FC
- Question: **how should we stack them?** Deeper? Wider? Which kernel sizes?
- Answered by the **ImageNet challenge (ILSVRC)**:
  - The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) evaluates algorithms for object detection and image classification at large scale
  - An annual computer vision competition that ran from 2010 to 2017 to benchmark algorithms in large-scale object classification and detection
  - **Dataset:** Built on a subset of the ImageNet database, primarily utilizing 1.2 million training images, 50,000 validation images, and 100,000 test images categorized into **1,000 distinct object and animal classes**. Currently the dataset has 14,197,122 images.
- Today: the key designs, and **the extra stage each one added**

ImageNet Progress: Top-5 Error

The metric shown is **Top-5 Error Rate (%)**, which measures how often the correct label is *not* among the model's top 5 predictions (lower is better).



- 2012: AlexNet first **deep CNN** wins by a huge margin
- Error halves again by 2015 as nets get **deeper**
- Human error is roughly **5%**; ResNet passes it

Timeline: What Each Model Added

Model	Year	Key idea added
LeNet-5	1998	Conv + pooling + FC, trained by backprop (digits)
AlexNet	2012	ReLU, dropout, GPU training, data augmentation; 8 layers
VGG	2014	Depth with only 3×3 convs; 16-19 layers
GoogLeNet / Inception	2014	Multi-scale branches, 1×1 bottlenecks; 22 layers, few params
ResNet	2015	Skip connections; 50-152 layers (Lecture 6)

AlexNet (2012)

ReLU

Replaced sigmoid/tanh: much **faster training**, less vanishing gradient.

Dropout + augmentation

Dropout on the FC layers and heavy image augmentation fought **overfitting** (≈60M parameters).

GPUs

Trained on 2 GPUs: showed that **big data + compute + depth** works.

Started the deep learning era.

VGG: Depth with Small Filters

- Oxford VGG, ILSVRC 2014 runner-up
- Design rule: **only 3×3 convs** (stride 1, same padding) and **2×2 max pools** (stride 2)
- Double the number of filters after each pool: 64 → 128 → 256 → 512
- Simple, uniform, easy to reuse; very popular **feature extractor**



Why Stack 3×3 Filters?

C denotes the input and output channel depth assuming $C_{\text{in}} = C_{\text{out}} = C$

Stack	Receptive field	Params (C channels or feature maps)	ReLUs
one 5×5	5×5	25 C^2	1
two 3×3	5×5	18 C^2 **(-28%)**	**2**
one 7×7	7×7	49 C^2	1
three 3×3	7×7	27 C^2 **(-45%)**	**3**

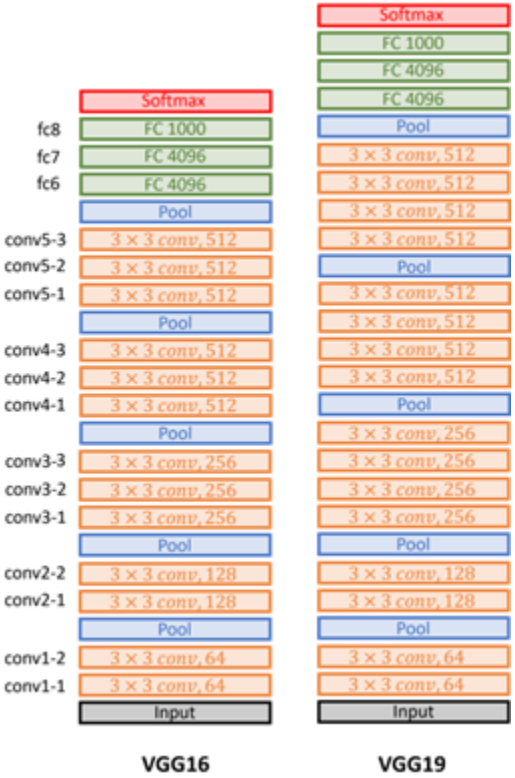
- Stacking three 3×3 convolutions covers the exact same 7×7 visual area as a single 7×7 filter.
- Reduces weights from 49 C^2 to 27 C^2, serving as a regularizer that lowers overfitting risk.
- Including 3 ReLUs instead of 1 creates a deeper, more expressive decision function.
- Standardizing on 3×3 Conv (stride 1, pad 1) allowed VGG to easily build deeper models (VGG-16 / VGG-19).

Key Takeaway: Stacking small 3×3 filters provides equal field of view with fewer weights and higher non-linear capacity.

VGG-16 Layer by Layer

Block	Layers	Filters	Output size
Input			224×224×3
Block 1	2 conv + pool	64	112×112×64
Block 2	2 conv + pool	128	56×56×128
Block 3	3 conv + pool	256	28×28×256
Block 4	3 conv + pool	512	14×14×512
Block 5	3 conv + pool	512	7×7×512
Classifier	FC 4096, FC 4096, FC 1000 + softmax		1000

VGG-16 vs VGG-19



- **16 vs 19** weight layers: VGG-19 adds one conv in blocks 3, 4, 5
- Parameters ≈ **138M (VGG-16)** vs **144M (VGG-19)**
- Accuracy gain is small (~0.1-0.5%);
  - Going deeper from 16 to 19 layers yields only a negligible improvement in ImageNet Top-5 error.
  - VGG-16 is used far more
- Deeper helped, but with **diminishing returns**

In deep learning and machine learning, **diminishing returns** describes a principle where **continually adding more resources (such as layers, parameters, compute, or training data) yields progressively smaller gains in performance**.

Where Are VGG's Parameters?

Convolutions

≈ **15M** parameters

but most of the **computation**

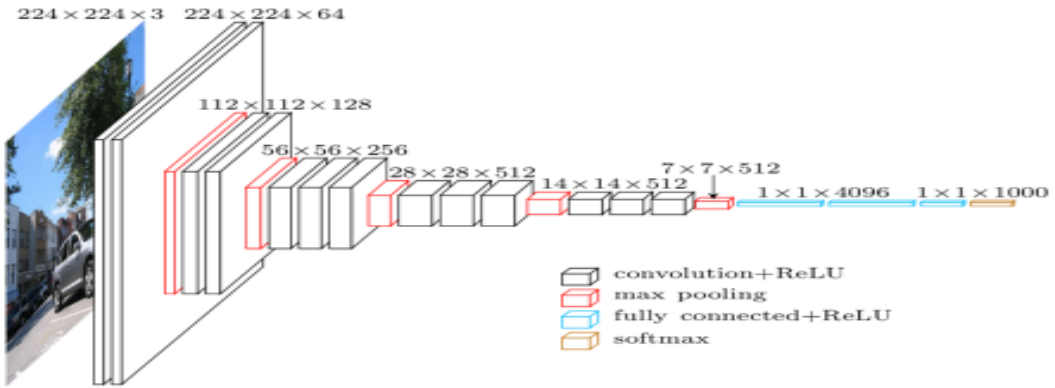
First FC layer

7×7×512×4096 ≈ **103M** parameters

about

75% of the whole network

Drawbacks: huge memory, slow, overfits. Later nets use global average pooling instead of big FC layers.



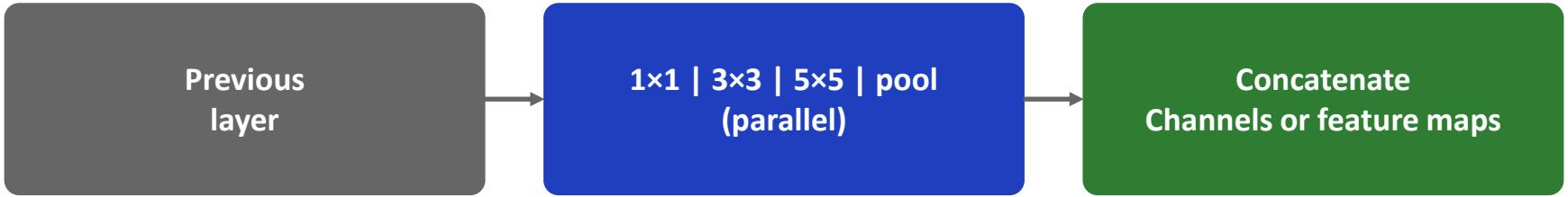
The Limits of "Just Go Deeper"

- More layers → more compute and memory
- **Vanishing gradients** obstruct the early layers (Lecture 3)
- Very deep plain nets got **worse**, not better (see ResNet)
- Idea: be **smarter** about how layers are connected, not just stack more

VGG CNNs Main Challenge

- A fundamental design challenge in VGG CNNs:
  - **Choosing the right filter size for multi-scale visual features.**
    - In real-world images, target objects appear at vastly different sizes (e.g., a tiny dog far away vs. a close-up dog face filling the frame).
    - **Small filters (**1×1**,** 3×3**)** excel at capturing local, fine-grained details and small features.
    - **Large filters (**5×5**)** cover a wider spatial receptive field, making them better suited for large or spread-out features.
    - **Standard Sequential CNNs (like VGG)** force the designer to manually pick a single filter size per layer (e.g., all 3×3)

Inception (e.g., GoogLeNet / ILSVRC 2014 winner): Look at Several Scales at Once



- Objects appear at **different sizes**: which kernel size is right?
- Inception answers: **use all of them in parallel** and let the network weigh them
- Outputs are concatenated along the channel (feature map) axis
    - **Padding is set to "same"** for every kernel size, ensuring every output feature map maintains identical spatial dimensions ($H \times W$).

The 1×1 Convolution Bottleneck

- A **1×1 conv** mixes channels at each pixel; it **reduces depth** cheaply
- Example: 28×28×192 input, want 28×28×32 feature maps with 5×5 kernels

Approach	Multiply-adds
Direct 5×5 conv, 32 filters	28×28×32×5×5×192 ≈ 120 M
1×1 reduce to 28×28×16, then 5×5, 32 filters: $\text{FLOPs}_1 = 28 \times 28 \times 16 \times (1 \times 1 \times 192) = 2{,}408{,}448 \approx \mathbf{2.4}$ M operations $\text{FLOPs}_2 = 28 \times 28 \times 32 \times (5 \times 5 \times 16) = 10{,}035{,}200 \approx \mathbf{10.0}$ M operations	2.4 M + 10.0 M ≈ 12.4 M

About 10× cheaper. The 1×1 trick is reused in ResNet bottlenecks.

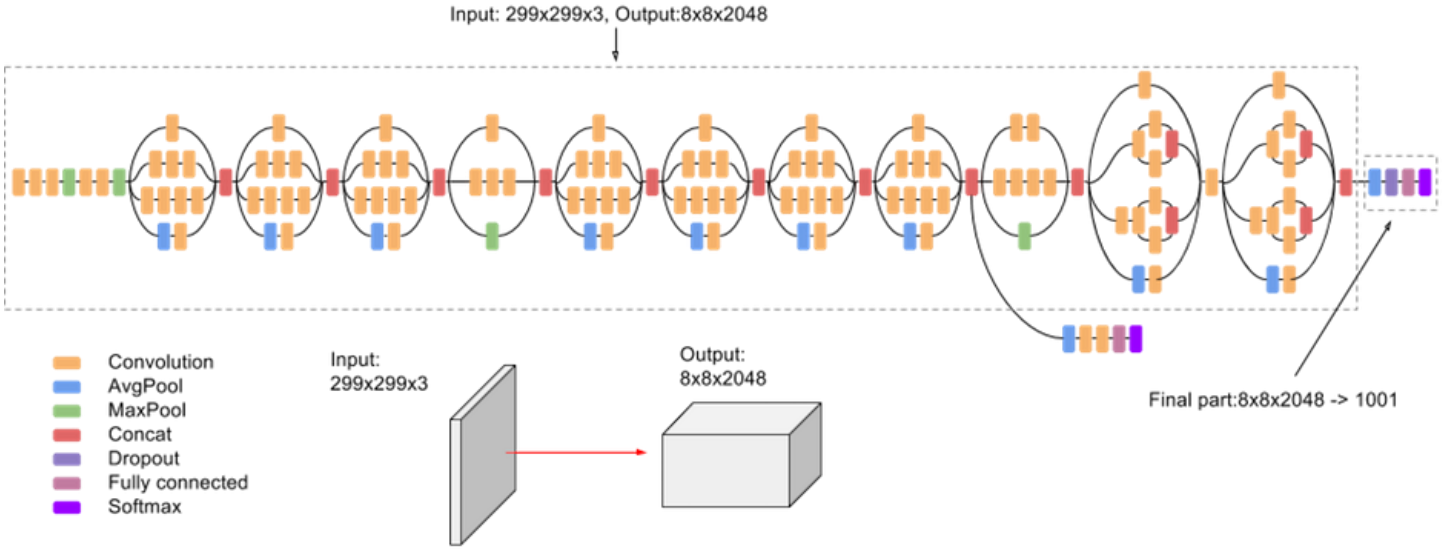
Key Takeaways

- $\sim 10\times$ **Computational Savings:** The bottleneck cuts total operations from 120M **down to** 12.4M (nearly a 90%reduction) while producing the exact same output spatial dimensions.
- **Foundational Architecture Trick:** This technique enabled GoogLeNet/Inception to go deeper without exploding computational requirements, and was later adopted directly in **ResNet bottleneck blocks** (ResNet-50/101/152) to efficiently process hundreds of channels.

GoogLeNet (Inception v1)

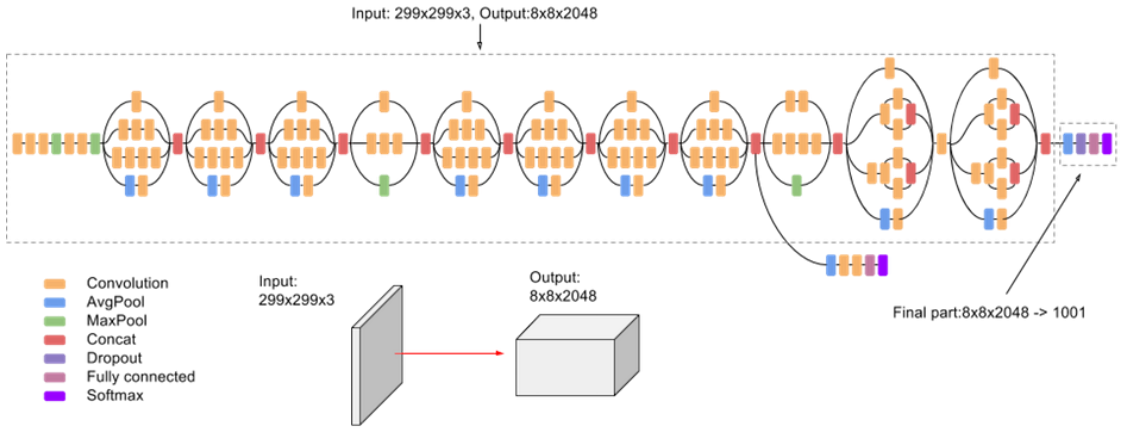
- 22 layers, only **≈7M parameters** (VGG: 138M)
- Stacked **Inception modules** with 1×1 bottlenecks
- **Global average pooling** instead of big FC layers
- Auxiliary classifiers during training to push gradients into the middle layers
- Won ILSVRC 2014 (≈6.7% top-5 error)

Inception v3: Cheaper and Better



- **Factorize** 5×5 → two 3×3; n×n → 1×n then n×1 (Recall Separable filters)
- **Label smoothing** and Batch Normalization (BN) in the auxiliary head

Inception v3: Global Average Pooling (GAP)



Instead of flattening the 3D feature tensor into a huge 1D vector, **Global Average Pooling** takes the average of *all pixels* across each individual feature map.

- **How it works:**
  - Suppose your final convolutional block outputs a tensor of shape $H \times W \times C$ (e.g., $7 \times 7 \times 512$).
  - For each of the C channels, GAP calculates a single scalar: the average value of its $H \times W$ grid ($7 \times 7 = 49$ spatial values).
  - The output is a simple vector of length C (e.g., $1 \times 1 \times 512$), representing 512 feature values.

$$\text{Input: } [H \times W \times C] \xrightarrow{\text{Global Average Pooling}} \text{Output: } [1 \times 1 \times C]$$

Feature	Inception v1 (GoogLeNet)	Inception v3
Input Shape	$224 \times 224 \times 3$	$299 \times 299 \times 3$
Large Kernels	Direct 5×5 convolutions	Factorized into two 3×3 convolutions
Spatial Factorization	Symmetric (3×3)	Asymmetric $(1 \times n$ followed by $n \times 1)$
Regularization	Dropout	Dropout + Label Smoothing
Efficiency Goal	Parallel multi-scale processing	Maximum parameter efficiency & reduced FLOPs
Total Layers with Learnable Weights	22	48

Side by Side

Model	Depth	Params	Top-5 err.*	Main idea
AlexNet	8	≈60M	15.3%	ReLU, dropout, GPU
VGG-16	16	≈138M	≈7.3%	3×3 stacks
GoogLeNet	22	≈7M	≈6.7%	Inception, 1×1
ResNet-152	152	≈60M	≈3.6%	Skip connections (next lecture)

* ILSVRC test results as reported in the papers (ensembles for some); values are approximate.

Note: An ensemble means here combining the predictions of multiple separately trained models (e.g., averaging the output probabilities of 6 slightly different versions of GoogLeNet or ResNet) rather than relying on a single neural network alone.

What Each Stage Added, and Why It Helps

Added stage	Problem it solves
ReLU	Vanishing gradients, slow training
Dropout, augmentation	Overfitting
Stacked 3×3 convs	Large receptive field with fewer weights
Inception branches	Unknown object scale
1×1 bottleneck	Compute and parameter cost
Global avg. pooling	Huge FC layers

Recap

- Depth plus small filters (**VGG**) improved accuracy
- **Inception** gained efficiency with multi-scale branches and 1×1 bottlenecks
- Parameters and compute are different budgets: VGG is heavy, GoogLeNet light
- Very deep plain networks still **degrade**
- Next: **ResNet**, transfer learning, and an intro to transformers